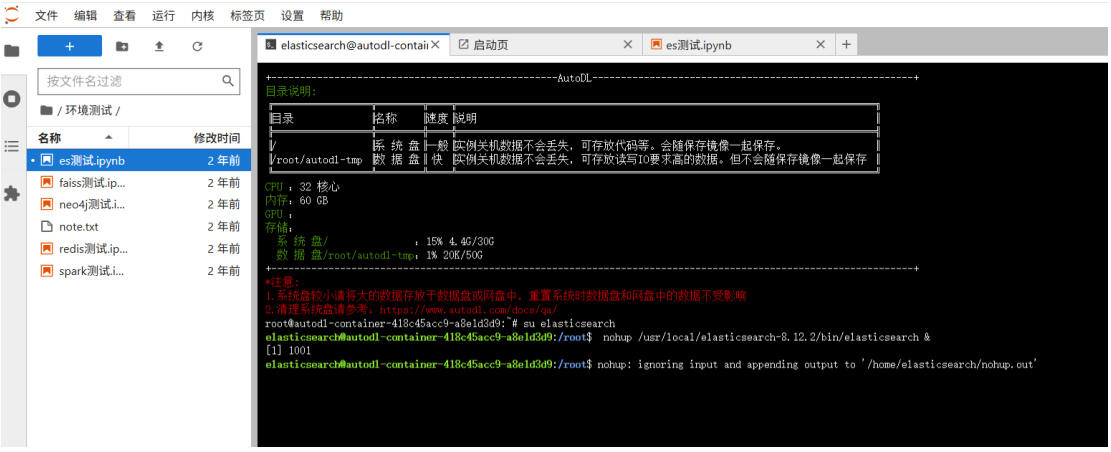
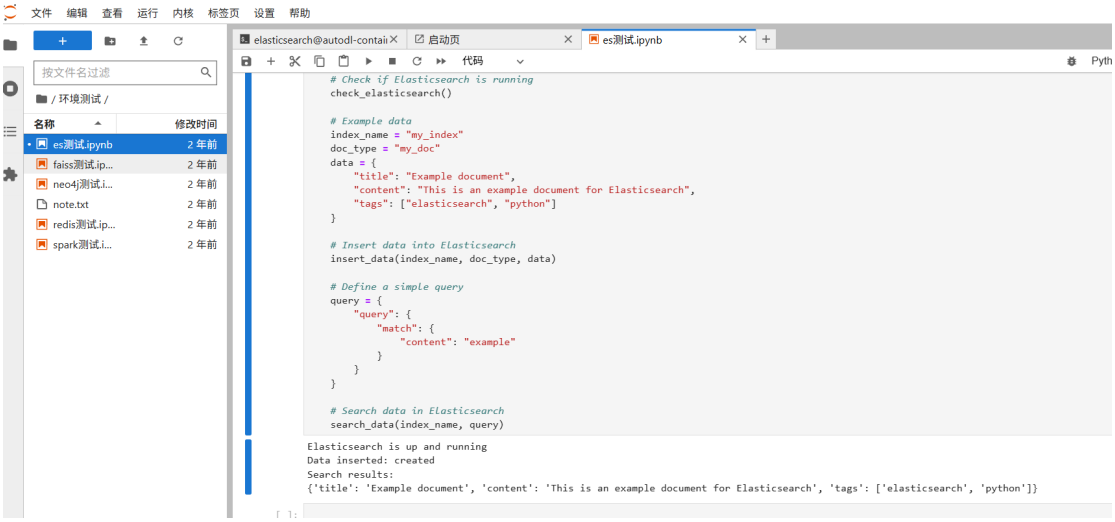
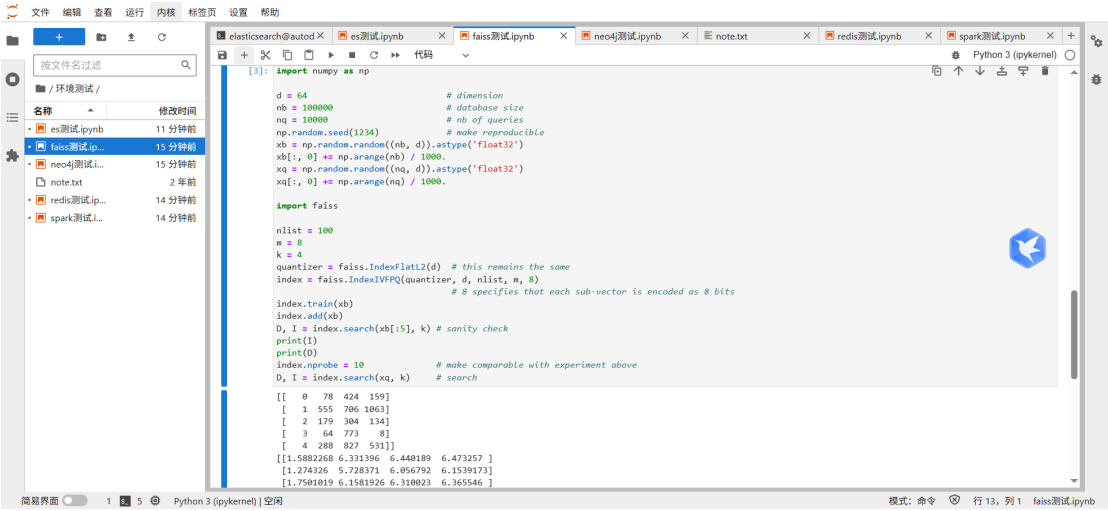
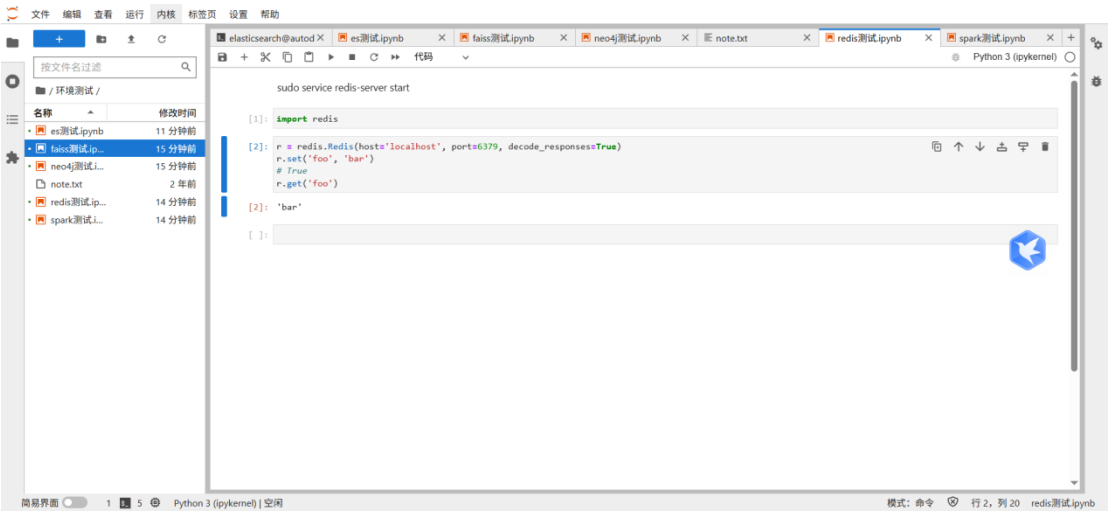
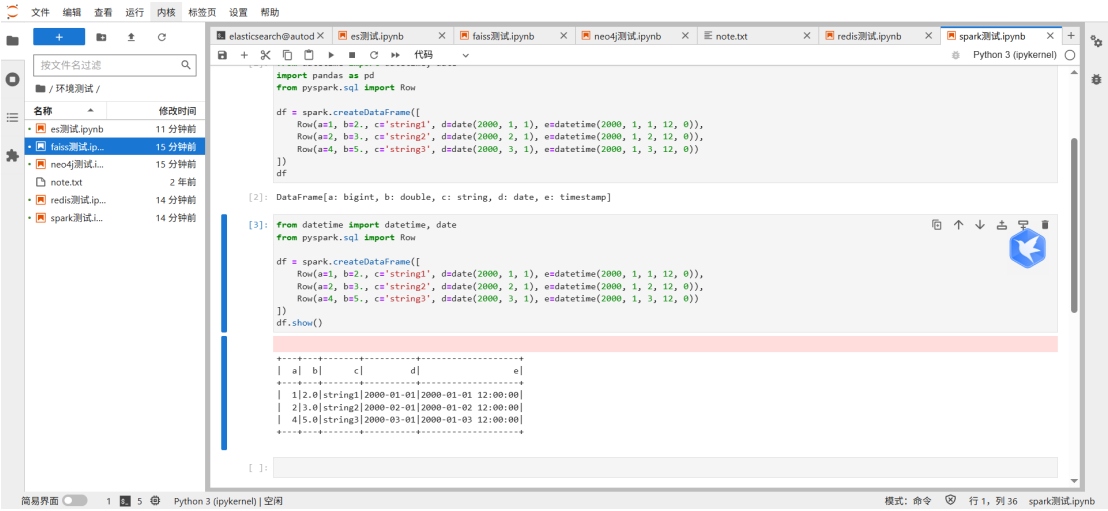
Task1:


文件 编辑 查看 运行 内核 标签页 设置 帮助
elasticsearch@autodl-contai
启动页
es测试.ipynb
按文件名过滤
/ 环境测试 /
名称
修改时间
es测试.ipynb 2 年前
faiss测试.ip... 2 年前
neo4j测试.i... 2 年前
note.txt 2 年前
redis测试.ip... 2 年前
spark测试.i... 2 年前
AutoDL
目录说明:
目录 名称 速度 说明
/ 系统盘 一般 实例关机数据不会丢失，可存放代码等。会随保存镜像一起保存。
/root/autodl-tmp 数据盘 快 实例关机数据不会丢失，可存放读写IO要求高的数据。但不会随保存镜像一起保存
CPU : 32 核心
内存: 60 GB
GPU :
存储:
系统盘/ : 15% 4.4G/30G
数据盘/root/autodl-tmp: 1% 20K/50G
*注意:
1.系统盘较小请将大的数据存放于数据盘或网盘中，重置系统时数据盘和网盘中的数据不受影响
2.清理系统盘请参考: https://www.autodl.com/docs/qa/
root@autodl-container-418c45acc9-a8e1d3d9:~# su elasticsearch
elasticsearch@autodl-container-418c45acc9-a8e1d3d9:/root$ nohup /usr/local/elasticsearch-8.12.2/bin/elasticsearch &
[1] 1001
elasticsearch@autodl-container-418c45acc9-a8e1d3d9:/root$ nohup: ignoring input and appending output to '/home/elasticsearch/nohup.out'


文件 编辑 查看 运行 内核 标签页 设置 帮助
elasticsearch@autodl-contai
启动页
es测试.ipynb
代码
# Check if Elasticsearch is running
check_elasticsearch()
# Example data
index_name = "my_index"
doc_type = "my_doc"
data = {
"title": "Example document",
"content": "This is an example document for Elasticsearch",
"tags": ["elasticsearch", "python"]
}
# Insert data into Elasticsearch
insert_data(index_name, doc_type, data)
# Define a simple query
query = {
"query": {
"match": {
"content": "example"
}
}
}
# Search data in Elasticsearch
search_data(index_name, query)
Elasticsearch is up and running
Data inserted: created
Search results:
{'title': 'Example document', 'content': 'This is an example document for Elasticsearch', 'tags': ['elasticsearch', 'python']}


文件 编辑 查看 运行 内核 标签页 设置 帮助
elasticsearch@autod
es测试.ipynb
faiss测试.ipynb
neo4j测试.ipynb
note.txt
redis测试.ipynb
spark测试.ipynb
代码
Python 3 (ipykernel)
[3]: import numpy as np
d = 64 # dimension
nb = 100000 # database size
nq = 10000 # nb of queries
np.random.seed(1234) # make reproducible
xb = np.random.random((nb, d)).astype('float32')
xb[:, 0] += np.arange(nb) / 1000.
xq = np.random.random((nq, d)).astype('float32')
xq[:, 0] += np.arange(nq) / 1000.
import faiss
nlist = 100
m = 8
k = 4
quantizer = faiss.IndexFlatL2(d) # this remains the same
index = faiss.IndexIVFPQ(quantizer, d, nlist, m, 8)
# 8 specifies that each sub-vector is encoded as 8 bits
index.train(xb)
index.add(xb)
D, I = index.search(xb[:5], k) # sanity check
print(I)
print(D)
index.nprobe = 10 # make comparable with experiment above
D, I = index.search(xq, k) # search
[[0 78 424 159]
[1 555 706 1063]
[2 179 304 134]
[3 64 773 8]
[4 288 827 531]]
[[1.5882268 6.331396 6.440189 6.473257]
[1.274326 5.728371 6.056792 6.1539173]
[1.7501019 6.1581926 6.310023 6.365546]


文件 编辑 查看 运行 内核 标签页 设置 帮助
elasticsearch@autod
es测试.ipynb
faiss测试.ipynb
neo4j测试.ipynb
note.txt
redis测试.ipynb
spark测试.ipynb
代码
Python 3 (ipykernel)
按文件名过滤
/ 环境测试 /
名称
修改时间
es测试.ipynb 11 分钟前
faiss测试.ip... 15 分钟前
neo4j测试.i... 15 分钟前
note.txt 2 年前
redis测试.ip... 14 分钟前
spark测试.i... 14 分钟前
sudo service redis-server start
[1]: import redis
[2]: r = redis.Redis(host='localhost', port=6379, decode_responses=True)
r.set('foo', 'bar')
# True
r.get('foo')
[2]: 'bar'
[ ]:
简易界面
Python 3 (ipykernel) | 空闲
模式：命令
行 2，列 20
redis测试.ipynb


文件 编辑 查看 运行 内核 标签页 设置 帮助
elasticsearch@autod
es测试.ipynb
faiss测试.ipynb
neo4j测试.ipynb
note.txt
redis测试.ipynb
spark测试.ipynb
代码
Python 3 (ipykernel)
按文件名过滤
/ 环境测试 /
名称
修改时间
es测试.ipynb 11 分钟前
faiss测试.ip... 15 分钟前
neo4j测试.i... 15 分钟前
note.txt 2 年前
redis测试.ip... 14 分钟前
spark测试.i... 14 分钟前
import pandas as pd
from pyspark.sql import Row
df = spark.createDataFrame([
Row(a=1, b=2., c='string1', d=date(2000, 1, 1), e=datetime(2000, 1, 1, 12, 0)),
Row(a=2, b=3., c='string2', d=date(2000, 2, 1), e=datetime(2000, 1, 2, 12, 0)),
Row(a=4, b=5., c='string3', d=date(2000, 3, 1), e=datetime(2000, 1, 3, 12, 0))
])
df
[2]: DataFrame[a: bigint, b: double, c: string, d: date, e: timestamp]
[3]: from datetime import datetime, date
from pyspark.sql import Row
df = spark.createDataFrame([
Row(a=1, b=2., c='string1', d=date(2000, 1, 1), e=datetime(2000, 1, 1, 12, 0)),
Row(a=2, b=3., c='string2', d=date(2000, 2, 1), e=datetime(2000, 1, 2, 12, 0)),
Row(a=4, b=5., c='string3', d=date(2000, 3, 1), e=datetime(2000, 1, 3, 12, 0))
])
df.show()
+---+---+-------+----------+-------------------+
| a| b| c| d| e|
+---+---+-------+----------+-------------------+
1	2.0	string1	2000-01-01	2000-01-01 12:00:00
2	3.0	string2	2000-02-01	2000-01-02 12:00:00
4	5.0	string3	2000-03-01	2000-01-03 12:00:00
+---+---+-------+----------+-------------------+
[ ]:
简易界面
Python 3 (ipykernel) | 空闲
模式：命令
行 1，列 36
spark测试.ipynb

Task2:

RAG101_01_读取数据集.py × {} questions.json

RAG101_01_读取数据集.py > ...

```python
import json  # 导入json模块，用于处理JSON数据

# pip install --upgrade charset-normalizer
# pip install pdfplumber
import pdfplumber  # 导入pdfplumber模块，用于处理PDF文件，内容读取

# 加载存储问题的JSON文件
questions = json.load(open("questions.json",encoding='utf-8'))

print("questions.json")  # 打印提示信息，显示加载的JSON文件名
print(questions[0])  # 打印第一个问题的内容

print("\n")  # 打印空行

# 打开PDF文件，单个pdf简单的内容解析，没有做格式识别
pdf = pdfplumber.open("汽车知识手册.pdf") # 以这个文件作为模拟的知识库
print("pages: ", len(pdf.pages))  # 打印提示信息，显示PDF文件的页数
```

问题 36　输出　调试控制台　终端 36　端口

终端

```
questions.json
{'question': '“前排座椅通风”的相关内容在第几页？', 'answer': '', 'reference': ''}


pages:  354
装载货物
前排储物空间 副仪表台
■ 遮阳板上的卡片/票据夹。 01无线充电板/置物板
■ 手套箱。
02杯托
■ 车门存储空间。
■ 眼镜盒。 03储物箱
■ 前副仪表台。
■ 前排中央扶手储物箱。 注意！
■ 切勿强行将尺寸不合适的容器卡入杯托。
24
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06>
```

RAG101_02_RAG检索TFIDF.py × {} questions.json

RAG101_02_RAG检索TFIDF.py > ...

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize

# 读取数据集
questions = json.load(open("questions.json",encoding='utf-8')) # 历史用户提问
pdf = pdfplumber.open("汽车知识手册.pdf") # 知识库
pdf_content = []
for page_idx in range(len(pdf.pages)): # 按照页面内容提取
    pdf_content.append({
        'page': 'page_' + str(page_idx + 1),
        'content': pdf.pages[page_idx].extract_text()
    })


# 对文本进行分词
question_words = [' '.join(jieba.lcut(x['question'])) for x in questions]
pdf_content_words = [' '.join(jieba.lcut(x['content'])) for x in pdf_content]
```

问题 53　输出　调试控制台　终端 53　端口

终端

```
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06> D:\ProgramData\miniconda3\envs\py312\python.exe "f:\11.八斗AI精品课\第2
: RAG工程化实现\Week06\RAG101_02_RAG检索TFIDF.py"
Building prefix dict from the default dictionary ...
Dumping model to file cache C:\Users\24478\AppData\Local\Temp\jieba.cache
Loading model cost 0.829 seconds.
Prefix dict has been built successfully.
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06> (D:\ProgramData\miniconda3\Scripts\activate) ; (conda activate py312)
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06>
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06>
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06>
```

RAG101_03_RAG检索BM25.py × {} questions.json

RAG101_03_RAG检索BM25.py > ...

rank_bm25 Aa ab .* 第 1 项，共 2 项

```python
from sklearn.preprocessing import normalize

# 读取数据集
questions = json.load(open("questions.json",encoding='utf-8'))
pdf = pdfplumber.open("汽车知识手册.pdf")
pdf_content = []
for page_idx in range(len(pdf.pages)):
    pdf_content.append({
        'page': 'page_' + str(page_idx + 1),
        'content': pdf.pages[page_idx].extract_text()
    })

# !pip install rank_bm25
from rank_bm25 import BM25Okapi

pdf_content_words = [jieba.lcut(x['content']) for x in pdf_content]
bm25 = BM25Okapi(pdf_content_words)

# 每个提问，与所有页面的bm25打分
for query_idx in range(len(questions)):
    # 只需要对部分的文档进行打分， 部分文档 =》 与提问包含了相同单词的文档 -》 从倒排
    doc_scores = bm25.get_scores(jieba.lcut(questions[query_idx]["question"]))
    max_score_page_idx = doc_scores.argsort()[::-1][0] + 1
    questions[query_idx]['reference'] = 'page_' + str(max_score_page_idx)

with open('submit_bm25_retrieval_top1.json', 'w', encoding='utf8') as up:
    json.dump(questions, up, ensure_ascii=False, indent=4)
```

问题 43 输出 调试控制台 终端 43 端口

终端

```
: RAG工程化实现\Week06\RAG101_03_RAG检索BM25.py"
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\24478\AppData\Local\Temp\jieba.cache
Loading model cost 0.855 seconds.
Prefix dict has been built successfully.
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06>
```

RAG101_04_RAG检索BERT.py × {} questions.json

RAG101_04_RAG检索BERT.py > ...

nbeddings-v2-base-zh Aa ab .* 第 2 项，共 3 项

```python
    score = feat @ pdf_embeddings.T
    max_score_page_idx = score.argsort()[::-1][0] + 1
    questions[query_idx]['reference'] = 'page_' + str(max_score_page_idx)

with open('submit_jina_retrieval_top1.json', 'w', encoding='utf8') as up:
    json.dump(questions, up, ensure_ascii=False, indent=4)

for query_idx, feat in enumerate(question_embeddings):
    score = feat @ pdf_embeddings.T
    max_score_page_idx = score.argsort()[::-1] + 1
    questions[query_idx]['reference'] = ['page_' + str(x) for x in max_score_page_idx[:10]]

with open('submit_jina_retrieval_top10.json', 'w', encoding='utf8') as up:
    json.dump(questions, up, ensure_ascii=False, indent=4)
```

问题 49 输出 调试控制台 终端 49 端口

终端

```
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06> D:\ProgramData\miniconda3\envs\py312\python.exe "f:\11.八斗AI精品课\第2期资料\week6\第6周
: RAG工程化实现\Week06\RAG101_04_RAG检索BERT.py"
D:\ProgramData\miniconda3\envs\py312\Lib\site-packages\transformers\models\bert\modeling_bert.py:440: UserWarning: 1Torch was not compiled with flash attenti
on. (Triggered internally at C:\cb\pytorch_1000000000000\work\aten\src\ATen\native\transformers\cuda\sdp_utils.cpp:555.)
  attn_output = torch.nn.functional.scaled_dot_product_attention(
Some weights of BertModel were not initialized from the model checkpoint at F:/11.八斗AI精品课/第2期资料/week6/第6周：RAG工程化实现/Week06/models/jinaai/jina
-embeddings-v2-base-zh/ and are newly initialized: ['embeddings.position_embeddings.weight', 'encoder.layer.0.intermediate.dense.bias', 'encoder.layer.0.inte
rmediate.dense.weight', 'encoder.layer.0.output.LayerNorm.bias', 'encoder.layer.0.output.LayerNorm.weight', 'encoder.layer.0.output.dense.bias', 'encoder.lay
er.0.output.dense.weight', 'encoder.layer.1.intermediate.dense.bias', 'encoder.layer.1.intermediate.dense.weight', 'encoder.layer.1.output.LayerNorm.bias', '
encoder.layer.1.output.LayerNorm.weight', 'encoder.layer.1.output.dense.bias', 'encoder.layer.1.output.dense.weight', 'encoder.layer.10.intermediate.dense.bi
as', 'encoder.layer.10.intermediate.dense.weight', 'encoder.layer.10.output.LayerNorm.bias', 'encoder.layer.10.output.LayerNorm.weight', 'encoder.layer.10.ou
tput.dense.bias', 'encoder.layer.10.output.dense.weight', 'encoder.layer.11.intermediate.dense.bias', 'encoder.layer.11.intermediate.dense.weight', 'encoder.
layer.11.output.LayerNorm.bias', 'encoder.layer.11.output.LayerNorm.weight', 'encoder.layer.11.output.dense.bias', 'encoder.layer.11.output.dense.weight', 'e
ncoder.layer.2.intermediate.dense.bias', 'encoder.layer.2.intermediate.dense.weight', 'encoder.layer.2.output.LayerNorm.bias', 'encoder.layer.2.output.LayerN
orm.weight', 'encoder.layer.2.output.dense.bias', 'encoder.layer.2.output.dense.weight', 'encoder.layer.3.intermediate.dense.bias', 'encoder.layer.3.intermed
```

RAG101_05_RAG检索BERT_Segment.py × questions.json

RAG101_05_RAG检索BERT_Segment.py > ...

```python

for page_idx in range(len(pdf.pages)):
    text = pdf.pages[page_idx].extract_text()
    for chunk_text in split_text_fixed_size(text, 40):
        pdf_content.append({
            'page': 'page_' + str(page_idx + 1),
            'content': chunk_text
        })

model = SentenceTransformer('F:/11.八斗AI精品课/第2期资料/week6/第6周：RAG工程化实现/Week06/models/BAAI/bge-small-zh-v1.5/')
question_sentences = [x['question'] for x in questions]
pdf_content_sentences = [x['content'] for x in pdf_content]

question_embeddings = model.encode(question_sentences, normalize_embeddings=True, show_progress_bar=True)
pdf_embeddings = model.encode(pdf_content_sentences, normalize_embeddings=True, show_progress_bar=True)

for query_idx, feat in enumerate(question_embeddings):
    score = feat @ pdf_embeddings.T
    max_score_page_idx = score.argsort()[-1]
```

问题 40 输出 调试控制台 终端 40 端口

终端

```
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06> D:\ProgramData\miniconda3\envs\py312\python.exe "f:\11.八斗AI精品课\第2期资料\week6\第6周
：RAG工程化实现\Week06\RAG101_05_RAG检索BERT_Segment.py"
Batches:   0%|                                                                                                    | 0/10 [00:00<?, ?it/s]
:\ProgramData\miniconda3\envs\py312\Lib\site-packages\transformers\models\bert\modeling_bert.py:440: UserWarning: 1Torch was not compiled with flash attenti
n. (Triggered internally at C:\cb\pytorch_1000000000000\work\aten\src\ATen\native\transformers\cuda\sdp_utils.cpp:555.)
  attn_output = torch.nn.functional.scaled_dot_product_attention(
Batches: 100%|[redacted]| 10/10 [00:00<00:00, 23.94it/s]
Batches: 100%|[redacted]| 117/117 [00:03<00:00, 35.34it/s]
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06>
```

RAG101_06_RAG重排序.py × RAG101_07_RAG多路召回.py RAG101_08_RAG问答.py questions.json

RAG101_06_RAG重排序.py > ...

print Aa ab .* 第 1 项

```python

fusion_result = []
k = 60 # 超参数，实现发现60
for q1, q2 in zip(bge, bm25):
    # q1 每个提问 的 bge 检索的top10
    # q2 每个提问 的 bm25 检索的top10
    print(len(fusion_result), len(bge))

    # 多路检索的 结果的合并
    fusion_score = {} # 每个页面 最终的打分
    for idx, q in enumerate(q1['reference']):
        if q not in fusion_score:
            fusion_score[q] = 1 / (idx + k) # 排在后面，得分更低
        else:
            fusion_score[q] += 1 / (idx + k)

    for idx, q in enumerate(q2['reference']):
```

问题 54 输出 调试控制台 终端 54 端口

终端

```
284 301
285 301
286 301
287 301
288 301
289 301
290 301
291 301
292 301
293 301
294 301
295 301
296 301
297 301
298 301
299 301
300 301
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06>
```

RAG101_06_RAG重排序.py | RAG101_07_RAG多路召回.py | RAG101_08_RAG问答.py 1 | questions.json

RAG101_07_RAG多路召回.py > ...

F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06\RAG101_07_RAG多路召回.py

```python
import json
bge = json.load(open('submit_bge_sgement_retrieval_top10.json',encoding='utf-8'))
bm25 = json.load(open('submit_bm25_retrieval_top10.json',encoding='utf-8'))

fusion_result = []
k = 60
for q1, q2 in zip(bge, bm25):
    fusion_score = {}
    for idx, q in enumerate(q1['reference']):
        if q not in fusion_score:
            fusion_score[q] = 1 / (idx + k)
        else:
            fusion_score[q] += 1 / (idx + k)

    for idx, q in enumerate(q2['reference']):
        if q not in fusion_score:
            fusion_score[q] = 1 / (idx + k)
        else:
            fusion_score[q] += 1 / (idx + k)
```

问题 55 | 输出 | 调试控制台 | 终端 54 | 端口

终端

```
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06> D:\ProgramData\miniconda3\envs\py312\python.exe "f:\11.八斗AI精品课\第2期资料\week6
：RAG工程化实现\Week06\RAG101_07_RAG多路召回.py"
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06> (D:\ProgramData\miniconda3\Scripts\activate) ; (conda activate py312)
PS F:\11.八斗AI精品课\第2期资料\week6\第6周：RAG工程化实现\Week06>
```

RAG101_06_RAG重排序.py | RAG101_07_RAG多路召回.py | RAG101_08_RAG问答.py | questions.json

RAG101_08_RAG问答.py > ...

```python

    prompt = '''你是一个汽车专家，你擅长编写和回答汽车相关的用户提问，帮我结合给定的资料，回答下面的问题。
如果问题无法从资料中获得，或无法从资料中进行回答，请回答无法回答。如果提问不符合逻辑，请回答无法回答。
如果问题可以从资料中获得，则请逐步回答。

资料：{0}


问题：{1}
    '''.format(reference_content, q1["question"])
    # answer = ask_glm(prompt)['choices'][0]['message']['content']

    answer = '无法'
    for _ in range(5):
        try:
            answer = ask_glm(prompt)['choices'][0]['message']['content']
            if answer:
                break
```

问题 54 | 输出 | 调试控制台 | 终端 54 | 端口

终端

```
车辆必须定期清洗。
■ 请尽快清除漆面上的污染物，如鸟粪、树汁、金属颗粒或小虫。
■ 请用湿海绵和大量的水洗掉污垢。请用专用车辆清洁剂彻底清洁
车辆。
■ 洗掉车辆下方的污垢。
■ 请勿清洗表面发烫或被阳光直射、暴晒下的车辆，需等待车身外
表冷却后再进行清洗。
警告！
如果您对胎面厚度不太了解，请联系Lynk&Co领克中心寻求帮助。 ■ 请勿用水冲洗发动机舱内部。
316
【模型回答】
结合给定的资料，无法回答问题。



【用户提问】
如何正确进行车辆的漆面保养以减少腐蚀风险？
```